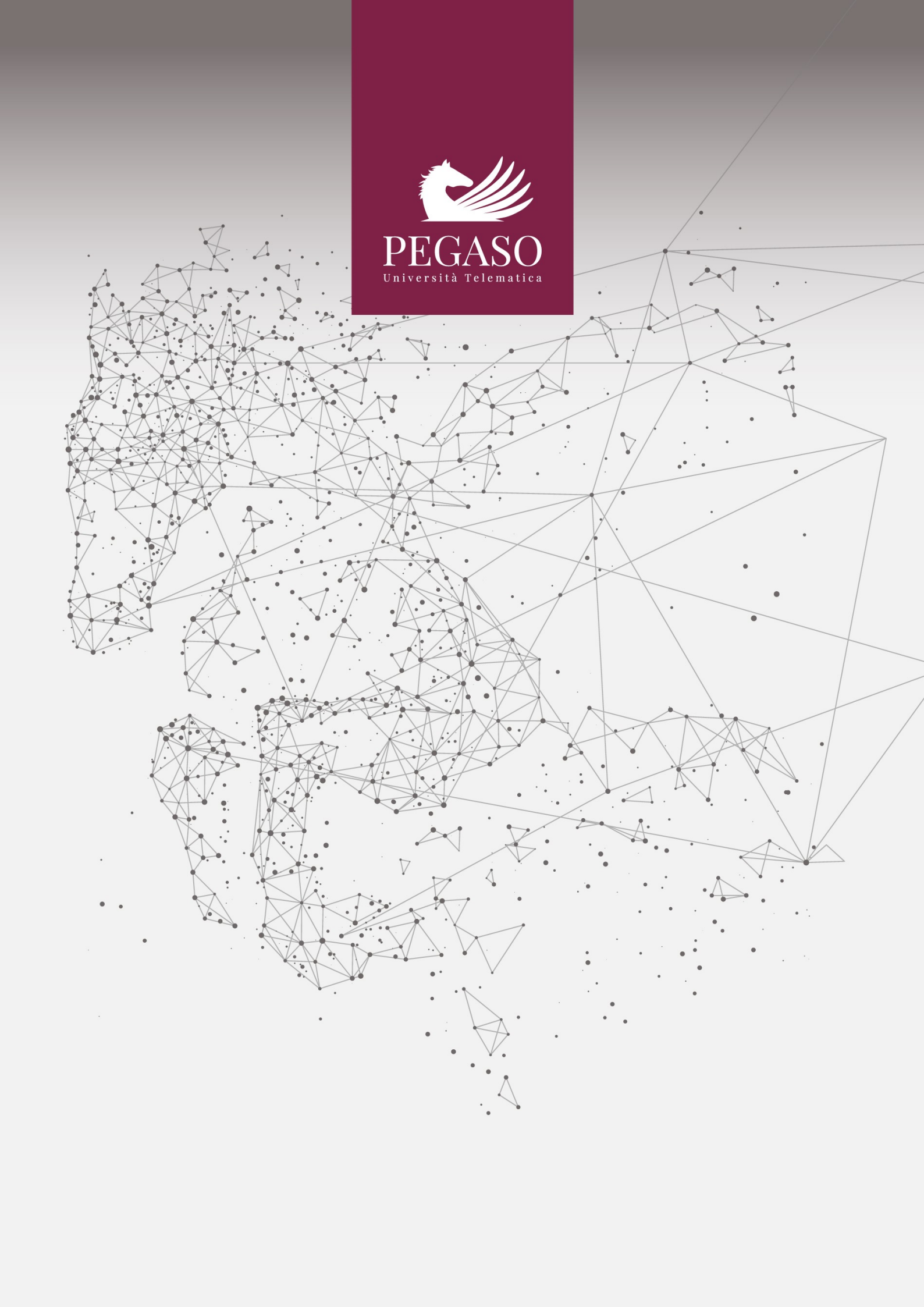

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. HASHING 3
2. TABELLA HASH 6
3. COME REALIZZARE UNA FUNZIONE HASH 8
BIBLIOGRAFIA 11

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Hashing

Tutte le volte in cui utilizziamo i dizionari, stiamo di fatto utilizzando l'hashing. Un dizionario è una struttura dati utilizzata per memorizzare insiemi dinamici di coppie:

$$< chiave, valore >$$

Le coppie sono indicizzate in base alla chiave, il valore è un dato satellite.

Il contesto nel quale ci stiamo muovendo è dunque il seguente:

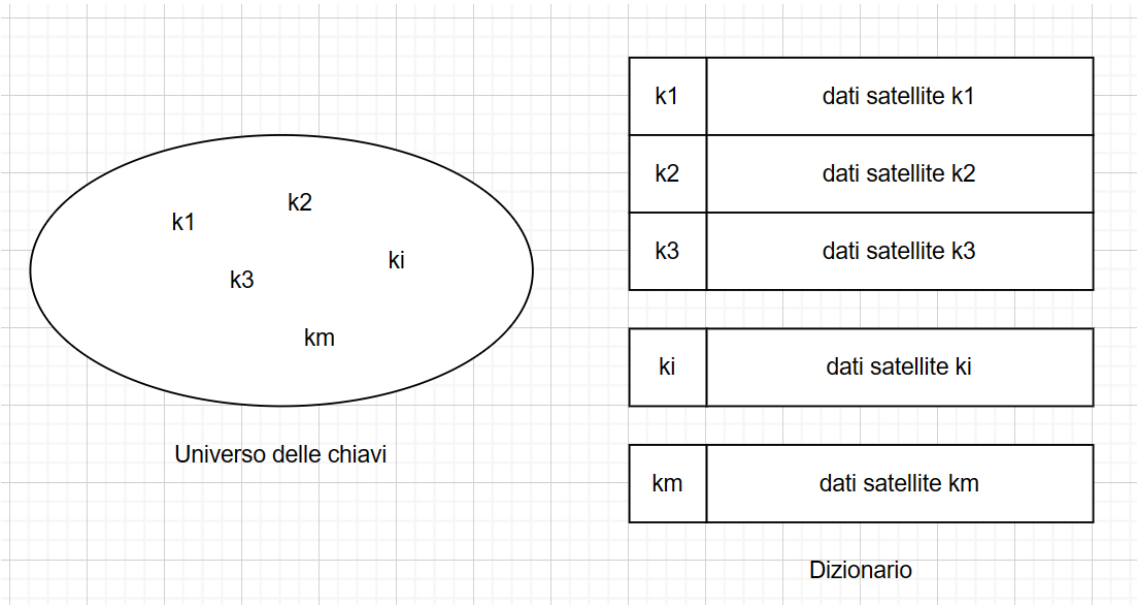


Figura 1: Dizionario ed Universo delle chiavi

Come si evince dalla figura, abbiamo dunque il nostro dizionario (l'insieme delle coppie $< chiave, valore >$) e l'universo delle chiavi, contenente semplicemente i valori delle chiavi (non sono inclusi in questo universo i dati satellite associati alle chiavi).

Le operazioni che si possono compiere su un dizionario sono le seguenti:

- lookup(key) → value
- insert(key, value)
- remove(key)

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un dizionario può essere implementato in modi differenti ed a seconda dell'implementazione che scelgo posso avere complessità differenti, in particolare:

	Array Unordered	Array Ordered	List	Red-Black Tree
`lookup(key)`	$O(n)$	$O(\log_2 n)$	$O(n)$	$O(\log_2 n)$
`insert(key, value)`	$O(1), O(n)$	$O(n)$	$O(1), O(n)$	$O(\log_2 n)$
`remove(key)`	$O(n)$	$O(n)$	$O(n)$	$O(\log_2 n)$
`foreach`	$O(n)$	$O(n)$	$O(n)$	$O(n)$

L'implementazione ideale è quella per cui ogni operazione (tranne la foreach) abbia una complessità $O(1)$.

La **tabella hash** nasce proprio con questo obiettivo.

Da un punto di vista "formale":

la **tabella hash** è un vettore $T[0 \dots m - 1]$ di dimensione m in cui sono memorizzate le chiavi di un insieme universo U di dimensione u. La modalità di memorizzazione è determinata da una **funzione hash** definita come:

$$h : U \to \{0, 1, \dots, m - 1\}$$

Data dunque una chiave $k \in U$, a questa viene associato un valore hash pari ad $h(k)$ e tale valore viene utilizzato per calcolare l'indice della tabella hash in cui memorizzare la chiave. In altre parole, la funzione hash converte la chiave in un intero che viene utilizzato per determinare l'indice della tabella hash in cui la chiave viene memorizzata.

ATTENZIONE: ci si potrebbe domandare il perché SE abbiamo u chiavi, la dimensione della tabella hash è pari ad m (verrebbe spontaneo immaginare che ad u chiavi corrispondano u elementi nella tabella hash); in realtà (come vedremo in seguito), generalmente vi sono delle "collisioni", cioè lo stesso indice della tabella hash identifica più chiavi.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

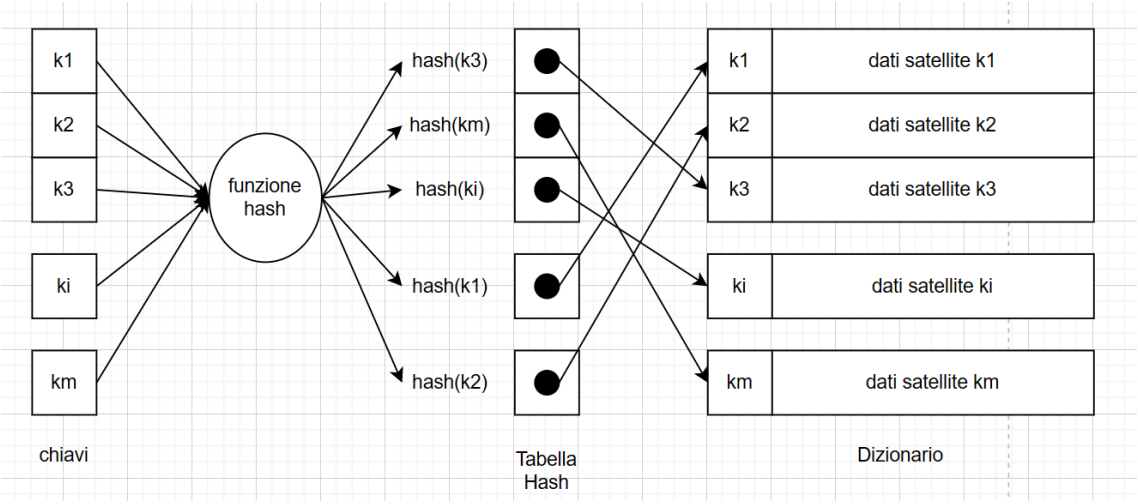


Figura 2: Principio di funzionamento

Supponiamo di avere una chiave key e di volerla memorizzare in una tabella hash di dimensione m. La funzione hash applicata alla chiave restituisce un intero compreso tra 0 e $m-1$: questo valore intero viene quindi utilizzato come indice della tabella hash in cui la chiave key viene memorizzata. In questo modo, quando si cerca la chiave key, la funzione hash viene applicata alla chiave e il valore hash restituito viene utilizzato per accedere direttamente alla posizione della tabella hash in cui la chiave è stata memorizzata; in realtà, nella tabella hash si ha tipicamente il puntatore all'oggetto identificato dalla chiave (in modo tale da poter accedere all'oggetto comprensivo dei dati satellite).

Intuiamo bene il vantaggio di tale strategia: si ha la possibilità di effettuare operazioni di ricerca e inserimento in tempo costante, ovvero in un tempo indipendente dal numero di elementi presenti nella tabella hash; ciò significa che la ricerca di un elemento all'interno della tabella hash richiede solo il calcolo del valore hash della chiave, senza la necessità di scorrere l'intera tabella per trovare l'elemento.

Come anticipato, quando due o più chiavi nel dizionario hanno lo stesso valore hash, diciamo che è avvenuta una **collisione**; idealmente, vorremmo funzioni hash senza collisioni.

Analizziamo alcuni scenari.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. TABELLA HASH

Il caso più semplice di tabella hash è quello di una tabella ad indirizzamento diretto. Quando l'universo delle chiavi è relativamente piccolo, la funzione hash di fatto fornisce come valore hash il valore stesso della chiave, ovvero sia la tabella hash contiene all'indice k il valore della chiave k.

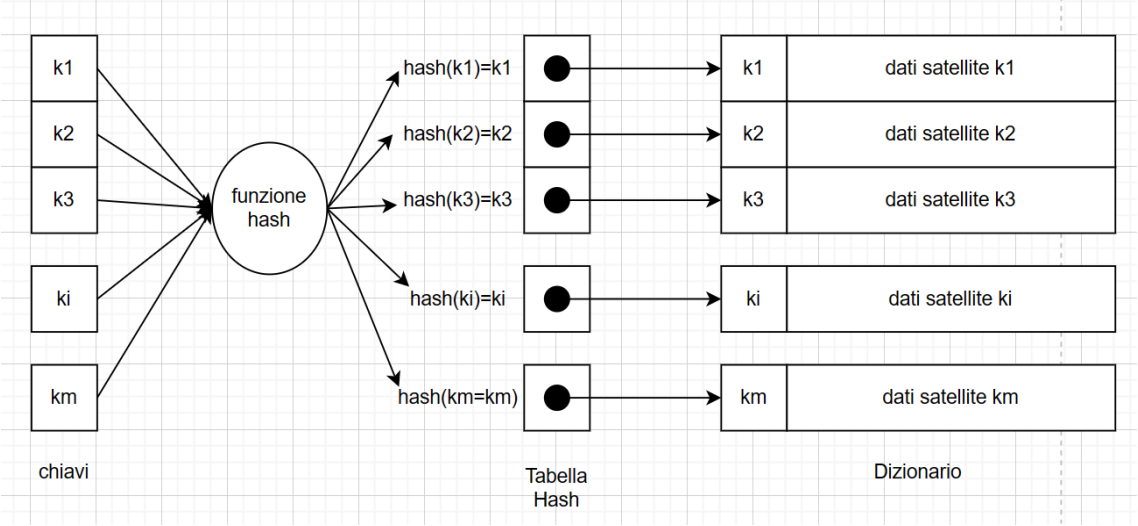


Figura 3: Tabella hash ad indirizzamento diretto

Capiamo bene che tale scenario ha senso solamente se l'insieme delle chiavi è "piccolo"; immaginiamo ad es.

- l'insieme dei giorni dell'anno (da 1 a 366)
- l'insieme dei mesi (da 1 a 12)
- l'insieme dei piloti in corsa al Giro d'Italia (numerati da 1 a 120, o all'ultimo numero in corsa)
- […]

La funzione di hash, in questo caso specifico è la **funzione hash identità**: $h(k) = k$

Per alcune applicazioni, gli elementi dell'insieme dinamico possono essere memorizzati nella stessa tabella a indirizzamento diretto, cioè, anzichè memorizzare la chiave e i dati satelliti di un elemento in un oggetto esterno alla tabella a indirizzamento diretto, con un puntatore da uno slot della tabella all'oggetto, possiamo memorizzare l'oggetto nello slot stesso, risparmiando spazio.

Inoltre, spesso non è necessario memorizzare la chiave dell'oggetto, in quanto se abbiamo l'indice di un oggetto nella tabella, abbiamo la sua chiave. Se le chiavi non sono memorizzate, però, dobbiamo disporre di qualche metodo per sapere se uno slot è vuoto.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La difficoltà dell'indirizzamento diretto è ovvia: se l'universo delle chiavi U è molto grande, memorizzare una tabella T di dimensione U può essere impraticabile, o perfino impossibile, considerando la memoria disponibile in un tipico calcolatore.

Inoltre, l'insieme K delle chiavi effettivamente memorizzate potrebbe essere così piccolo rispetto a U che la maggior parte dello spazio allocato per la tabella T sarebbe sprecato.

Quando l'insieme K delle chiavi memorizzate in un dizionario è molto più piccolo dell'universo U di tutte le chiavi possibili, una tabella hash richiede molto meno spazio di una tabella a indirizzamento diretto.

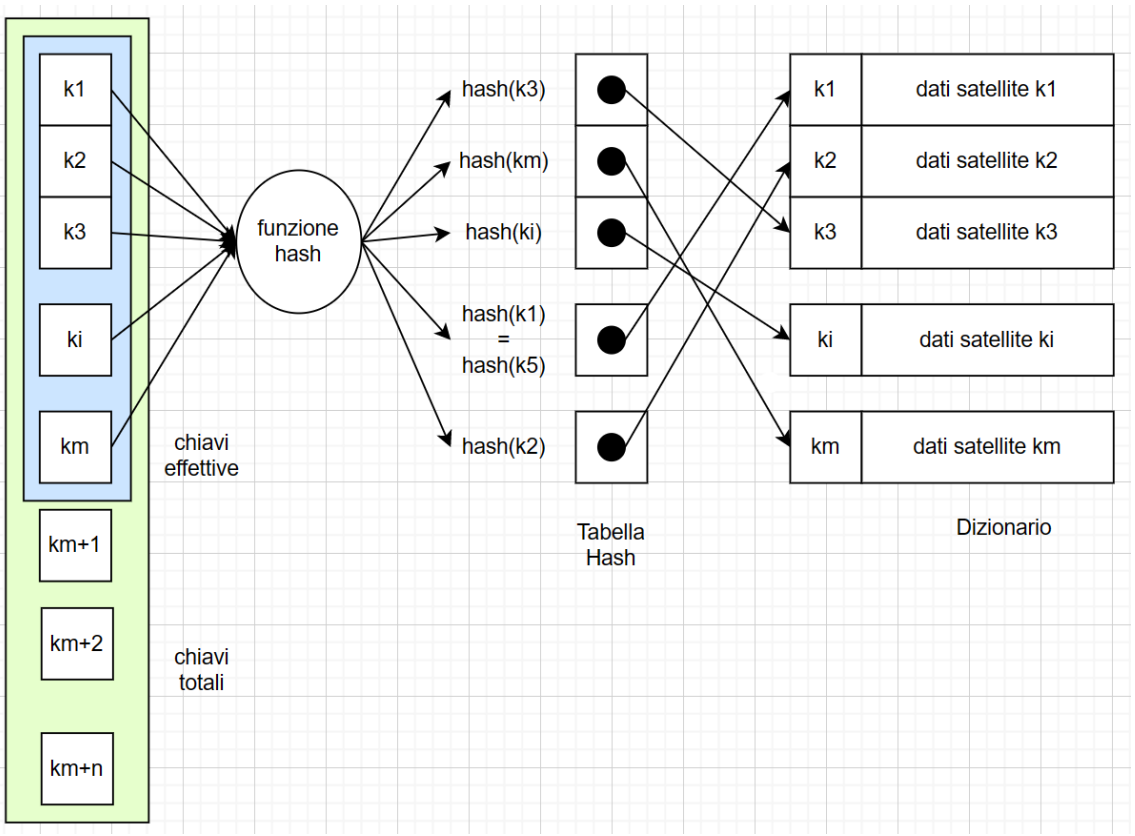


Figura 4: Chiavi effettive

Come si evince dalla figura, questa volta abbiamo un sottoinsieme di chiavi (le chiavi effettive identificate dall'insieme celeste) che saranno oggetto di hashing; vi sarà un insieme di chiavi che esisteranno ma che non essendo effettivamente utilizzate, resteranno fuori dalla procedura di hashing (quelle dell'insieme verde che non rientrano nell'insieme celeste).

Come si evince dallo schema, è anche possibile che vi siano delle collisioni, cioè l'hash di due o più chiavi può essere lo stesso, identificando la stessa posizione nella tabella hash: nel nostro esempio abbiamo che la chiave k1 e k5 hanno lo stesso hash e quindi indentificano la stessa posizione nella tabella.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Come realizzare una funzione hash

Una funzione hash h si dice **perfetta** se è **iniettiva**, ovvero se non dà origine a collisioni:

$$\forall k_1, k_2 \in U: k_1 \neq k2 \Rightarrow h(k1) \neq h(k2)$$

È spesso impraticabile ottenere una funzione hash perfetta: lo spazio delle chiavi è spesso grande, sparso, non conosciuto. Se non possiamo evitare le collisioni almeno cerchiamo di minimizzare il loro numero: vogliamo funzioni che distribuiscano uniformemente le chiavi negli indici $[0 \dots m - 1]$ della tabella hash.

Se $P(k)$ indica la probabilità che una chiave k sia inserita in tabella e $Q(i)$ la probabilità che una chiave finisca nella cella i:

$$Q(i) = \sum_{k \in U: h(K)=i} P(k)$$

una funzione hash h gode di **uniformità semplice** se:

$$\forall i \in [0,\dots,m-1]: Q(i) = 1/m$$

Per poter ottenere una funzione hash con uniformità semplice, la distribuzione delle probabilità P deve essere nota; la distribuzione esatta può non essere (completamente) nota: si utilizzano allora tecniche "euristiche".

Teniamo presente che in generale:

- l'iniettività è un ideale;
- possiamo "avvicinarci" all'uniformità semplice;
- in generale quello che vogliamo fare è "minimizzare i problemi" e cioè vogliamo cercare di avere "poche collisioni".

Vediamo da un punto di vista operativo alcuni esempi di funzione di hash che possiamo costruire. Partiamo dal presupposto che le chiavi possono comunque essere tradotte in valori numerici, anche interpretando la loro rappresentazione in memoria come un numero.

Ad esempio, consideriamo la trasformazione di stringhe:

- ord(c): valore ordinale binario del carattere c in qualche codifica;
- bin(k): rappresentazione binaria della chiave k, concatenando i valori binari dei caratteri che lo compongono;
- int(b): valore numerico associato al numero binario b;
- int(k) = int(bin(k)).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Possiamo considerare un codice ASCII ad 8 bit:

$$bin(\text{“}DOG\text{”}) = ord(\text{“}D\text{”})\ ord(\text{“}O\text{”})\ ord(\text{“}G\text{”}) = 01000100\ 01001111\ 01000111$$

$$int(\text{“}DOG\text{”}) = 68 \cdot 2562 + 79 \cdot 256 + 71 = 4476743$$

Per facilitare i "calcolo" possiamo usare un tool online: Text to Binary Converter (rapidtables.com)

Se pensassimo di operare tramite "l'estrazione" di alcuni BIT possiamo in ogni caso avere dei problemi:

- Estrazione dei bit **meno significativi**:
  - $m = 2^p = 2^{16} = 65536$ →16 bit meno significativi di $bin(k)$
  - $bin(\text{“}suono\text{”}) = 01110011\ 01110101\ 01101111\ 01101110\ 01101111$
  - $bin(\text{“}tuono\text{”}) = 01110100\ 01110101\ 01101111\ 01101110\ 01101111$
  - $H(\text{“}suono\text{”}) = int(0110111001101111) = 28271$
  - $H(\text{“}tuono\text{”}) = int(0110111001101111) = 28271$
  - Come si evince, l'hash di suono e tuono è identico e quindi si ha la collisione
- Estrazione dei bit **all'interno** della parola:
  - $m = 2^p = 2^{16} = 65536$ →16 bit all'interno di $bin(k)$
  - $bin(\text{“}suono\text{”}) = 01110011\ 01110101\ 01101111\ 01101110\ 01101111$
  - $bin(\text{“}tuono\text{”}) = 01110100\ 01110101\ 01101111\ 01101110\ 01101111$
  - $H(\text{“}suono\text{”}) = int(101010110111101) = 21949$
  - $H(\text{“}tuono\text{”}) = int(101010110111101) = 21949$
  - Come si evince, l'hash di suono e tuono è identico e quindi si ha la collisione
- XOR: somma modulo 2, (bit a bit), di sottoinsiemi di p bit di $bin(k)$
  - Se ho parole che sono "permutazioni" l'una dell'altra incorro nello stesso problema
- Metodo della divisione $modulo\ m$ (con m numero primo)
  - $H(k) = int(k)\ mod\ m$; ad es. $m = 383$
  - $bin(\text{“}suono\text{”}) = 01110011\ 01110101\ 01101111\ 01101110\ 01101111$
  - $bin(\text{“}tuono\text{”}) = 01110100\ 01110101\ 01101111\ 01101110\ 01101111$
  - $dec(bin(\text{“}suono\text{”})) = 495891476079\ mod\ 383 = 83$
  - $dec(bin(\text{“}tuono\text{”})) = 500186443375\ mod\ 383 = 17$

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ATTENZIONE:

- l'operazione di modulo di una potenza di 2, ad es. 2^k, comporta l'estrazione degli ultimi k bit della mia sequenza binaria (dei k bit meno significativi) e pertanto sarebbe un problema; occorre dunque evitare che $m = 2^k$;
- occorre evitare che $m = 2^{k-1}$, infatti permutazioni di stringhe con set di caratteri di dimensione 2^k hanno lo stesso valore hash;
- è bene scegliere dunque numeri primi distanti da potenze di 2 (e di 10).

Alcune delle funzioni hash più usate sono:

- **Nome Note Link**: FNV Hash Funzione hash non crittografica, creata nel 1991;
- **Murmur Hash**: Funzione hash non crittografica, creata nel 2008, il cui uso è ormai sconsigliato perchè debole;
- **City Hash**: Una famiglia di funzioni hash non-crittografiche, progettate da Google per essere molto veloce. Ha varianti a 32, 64, 128, 256 bit;
- **Farm Hash**: Il successore di City Hash, sempre sviluppato da Google.

Vi è poi la famiglia delle funzioni Hash crittografiche (SHA-1) che hanno un preciso obiettivo: deve essere molto difficile o quasi impossibile risalire al testo che ha portato ad un dato hash.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione;
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione;
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley;
- Sedgewick: Algoritmi in C, Pearson, 2015;
- Cormen Leiserson Rivest Stein-Introduzione Agli Algoritmi E Strutture Dati-Prima Edizione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).